Elias-Fano coding

Advanced Data Structures – Summer 2026 – Lecture 2

Ragnar {Groot Koerkamp}, Stefan Walzer, Stefan Hermann

2026-04-27 Mon 14:00

Previous slides: Bit vectors, Rank & Select

- Bit vector: storing bits
- Rank: counting 1 bits
- Select: finding 1 bits

Goal: Storing lists of integers

How do we efficiently store a list of integers

$$0 \leq x_0 \leq x_1 \leq \cdots \leq x_{n-1} < U.$$

Theorem 1: ***Lower bound*.**

Using *stars and bars*, there are $\binom{U+n-1}{n}$ of choosing n elements (with duplicates) from $\{0, 1, \ldots, U-1\}$.

Goal: Storing lists of integers

How do we efficiently store a list of integers

$$0 \leq x_0 \leq x_1 \leq \cdots \leq x_{n-1} < U.$$

Theorem 1: ***Lower bound*.**

Using *stars and bars*, there are $\binom{U+n-1}{n}$ of choosing n elements (with duplicates) from $\{0, 1, \ldots, U-1\}$.

$$n\log_2\left(\frac{U+n-1}{n}\right) \leq \log_2\binom{U+n-1}{n} \leq n\log_2\left(e \cdot \frac{U+n-1}{n}\right)$$

so it can be done with $\log_2 e + \log_2\left(\frac{U+n-1}{n}\right)$ bits per key.

Goal: Storing lists of integers

How do we efficiently store a list of integers

$$0 \leq x_0 \leq x_1 \leq \cdots \leq x_{n-1} < U.$$

Theorem 1: ***Lower bound*.**

Using *stars and bars*, there are $\binom{U+n-1}{n}$ of choosing n elements (with duplicates) from $\{0, 1, \ldots, U-1\}$.

$$n\log_2\left(\frac{U+n-1}{n}\right) \leq \log_2\binom{U+n-1}{n} \leq n\log_2\left(e \cdot \frac{U+n-1}{n}\right)$$

so it can be done with $\log_2 e + \log_2\left(\frac{U+n-1}{n}\right)$ bits per key.

For $n = o(\sqrt{U})$ and $n \to \infty$,

$$\frac{1}{n}\log_2\binom{U+n-1}{n} \to \log_2\left(e \cdot \frac{U}{n}\right).$$

Storing many small integers ($n \gg U$)

Question 1.

What if $n \gg U$?

Answer 1.

Storing many small integers ($n \gg U$)

Question 1.

What if $n \gg U$?

Answer 1.

When $n \gg U$, store how often each element each element occurs.

Storing many small integers ($n \gg U$)

Question 1.

What if $n \gg U$?

Answer 1.

When $n \gg U$, store how often each element each element occurs.

- $U \log_2 n$ bits

Storing dense lists ($n \approx U$)

Question 2.

What if $n \approx U$?

(Hint: What if all elements are distinct?)

Answer 2.

Storing dense lists ($n \approx U$)

Question 2.

What if $n \approx U$?

(Hint: What if all elements are distinct?)

Answer 2.

When all elements are distinct: store U bits indicating for each 1 or 0.

Storing dense lists ($n \approx U$)

Question 2.

What if $n \approx U$?

(Hint: What if all elements are distinct?)

Answer 2.

When all elements are distinct: store U bits indicating for each 1 or 0.

- n bits

Storing dense lists ($n \approx U$)

Question 2.

What if $n \approx U$?

(Hint: What if all elements are distinct?)

Answer 2.

When all elements are distinct: store U bits indicating for each 1 or 0.

- n bits

Otherwise: encode all the counts in unary and concatenate them, e.g., a count of 3 becomes `1110`.

Storing dense lists ($n \approx U$)

Question 2.

What if $n \approx U$?

(Hint: What if all elements are distinct?)

Answer 2.

When all elements are distinct: store U bits indicating for each 1 or 0.

- n bits

Otherwise: encode all the counts in unary and concatenate them, e.g., a count of 3 becomes `1110`.

- $U + n$ bits

Storing dense lists ($n \approx U$)

Question 2.

What if $n \approx U$?

(Hint: What if all elements are distinct?)

Answer 2.

When all elements are distinct: store U bits indicating for each 1 or 0.

- n bits

Otherwise: encode all the counts in unary and concatenate them, e.g., a count of 3 becomes `1110`.

- $U + n$ bits
- Lower bound for $n = u$:

$$n\log_2\binom{U+n-1}{n} = n\log_2\binom{2n-1}{n} = n\log_2\left(\Theta(2^{2n}/\sqrt{n})\right) \approx 2n$$

Storing sparse lists ($n \ll U$)

Question 3.

What if $n \ll U$?

Answer 3.

Question 4.

Answer 4.

Storing sparse lists ($n \ll U$)

Question 3.

What if $n \ll U$?

Answer 3.

Store directly:

Question 4.

Answer 4.

Storing sparse lists ($n \ll U$)

Question 3.

What if $n \ll U$?

Answer 3.

Store directly:

- $n \log_2 U$ bits.

Question 4.

Answer 4.

Storing sparse lists ($n \ll U$)

Question 3.

What if $n \ll U$?

Answer 3.

Store directly:

- $n\log_2 U$ bits.
- Lower bound for $n \ll u$: $n\log_2\left(e \cdot \frac{U}{n}\right) \approx n\log_2(eU/n) = n(\log_2 U + \log_2 e - \log_2 n)$
  - $\log_2 n - \log_2 e$ bits/key wasted!

Question 4.

Answer 4.

Storing sparse lists ($n \ll U$)

Question 3.

What if $n \ll U$?

Answer 3.

Store directly:

- $n\log_2 U$ bits.
- Lower bound for $n \ll u$: $n\log_2\left(e \cdot \frac{U}{n}\right) \approx n\log_2(eU/n) = n(\log_2 U + \log_2 e - \log_2 n)$
  - $\log_2 n - \log_2 e$ bits/key wasted!

Question 4.

Where is the "waste"/inefficiency?

Answer 4.

Storing sparse lists ($n \ll U$)

Question 3.

What if $n \ll U$?

Answer 3.

Store directly:

- $n\log_2 U$ bits.
- Lower bound for $n \ll u$: $n\log_2\left(e \cdot \frac{U}{n}\right) \approx n\log_2(eU/n) = n(\log_2 U + \log_2 e - \log_2 n)$
  - $\log_2 n - \log_2 e$ bits/key wasted!

Question 4.

Where is the "waste"/inefficiency?

Answer 4.

Consecutive elements share the $\approx \log_2 n$ high bits!

Elias-Fano coding (Elias 1974; Fano 1971; Vigna 2013)

Theorem 2: ***Elias-Fano (EF) coding*.**

A list of n integers $0 \leq x_0 \leq x_1 \leq \cdots \leq x_{n-1} < U$ can be stored in

$$n(\log_2(U/n) + 2)$$

bits while supporting $O(1)$ access time.

This is only $2 - \log_2(e) = 0.5573$ bits/key away from the lower bound when $n = o(\sqrt{U})$!

Elias-Fano coding (Elias 1974; Fano 1971; Vigna 2013)

Theorem 2: ***Elias-Fano (EF) coding*.**

A list of n integers $0 \leq x_0 \leq x_1 \leq \cdots \leq x_{n-1} < U$ can be stored in

$$n(\log_2(U/n) + 2)$$

bits while supporting $O(1)$ access time.

This is only $2 - \log_2(e) = 0.5573$ bits/key away from the lower bound when $n = o(\sqrt{U})$!

Idea:

Elias-Fano coding (Elias 1974; Fano 1971; Vigna 2013)

Theorem 2: ***Elias-Fano (EF) coding*.**

A list of n integers $0 \leq x_0 \leq x_1 \leq \cdots \leq x_{n-1} < U$ can be stored in

$$n(\log_2(U/n) + 2)$$

bits while supporting $O(1)$ access time.

This is only $2 - \log_2(e) = 0.5573$ bits/key away from the lower bound when $n = o(\sqrt{U})$!

Idea:

- We have budget to store the $\ell = \lfloor \log(U/n) \rfloor$ bits explicitly.

Elias-Fano coding (Elias 1974; Fano 1971; Vigna 2013)

Theorem 2: *Elias-Fano (EF) coding*.

A list of n integers $0 \leq x_0 \leq x_1 \leq \cdots \leq x_{n-1} < U$ can be stored in

$$n(\log_2(U/n) + 2)$$

bits while supporting $O(1)$ access time.

This is only $2 - \log_2(e) = 0.5573$ bits/key away from the lower bound when $n = o(\sqrt{U})$!

Idea:

- We have budget to store the $\ell = \lfloor \log(U/n) \rfloor$ bits explicitly.
- But how do we encode all $\log U - \ell \approx \log n$ high bits using ≈ 2 bits/key?

Elias-Fano coding (Elias 1974; Fano 1971; Vigna 2013)

Theorem 2: *Elias-Fano (EF) coding*.

A list of n integers $0 \leq x_0 \leq x_1 \leq \cdots \leq x_{n-1} < U$ can be stored in

$$n(\log_2(U/n) + 2)$$

bits while supporting $O(1)$ access time.

This is only $2 - \log_2(e) = 0.5573$ bits/key away from the lower bound when $n = o(\sqrt{U})$!

Idea:

- We have budget to store the $\ell = \lfloor \log(U/n) \rfloor$ bits explicitly.
- But how do we encode all $\log U - \ell \approx \log n$ high bits using ≈ 2 bits/key?
- Using the $n \approx U$ case!

Elias-Fano coding

Definition 1.

Split each integer into a $\ell = \lfloor \log(U/n) \rfloor$-bit *low* part and $(\log U - \ell)$-bit *high* part.

- Store all low bits explicitly in $n\ell$ bits.

Elias-Fano coding

Definition 1.

Split each integer into a $\ell = \lfloor \log(U/n) \rfloor$-bit *low* part and $(\log U - \ell)$-bit *high* part.

- Store all low bits explicitly in $n\ell$ bits.
- Write how often each high part occurs in unary.
  - $\lceil U/2^\ell \rceil$ high parts with

$$n \leq \lceil U/2^\ell \rceil < 2n.$$

  - In total $n + \lceil U/2^\ell \rceil \leq 3n$ bits.

Elias-Fano coding

Definition 1.

Split each integer into a $\ell = \lfloor \log(U/n) \rfloor$-bit *low* part and $(\log U - \ell)$-bit *high* part.

- Store all low bits explicitly in $n\ell$ bits.
- Write how often each high part occurs in unary.
  - $\lceil U/2^\ell \rceil$ high parts with

$$n \leq \lceil U/2^\ell \rceil < 2n.$$

  - In total $n + \lceil U/2^\ell \rceil \leq 3n$ bits.
- Thus, space usage below

$$n\ell + 3n = n(\lfloor \log(U/n) \rfloor + 3)$$

Elias-Fano coding

Definition 1.

Split each integer into a $\ell = \lfloor \log(U/n) \rfloor$-bit *low* part and $(\log U - \ell)$-bit *high* part.

- Store all low bits explicitly in $n\ell$ bits.
- Write how often each high part occurs in unary.
  - $\lceil U/2^\ell \rceil$ high parts with

$$n \leq \lceil U/2^\ell \rceil < 2n.$$

  - In total $n + \lceil U/2^\ell \rceil \leq 3n$ bits.
- Thus, space usage below

$$n\ell + 3n = n(\lfloor \log(U/n) \rfloor + 3)$$

- More refined analysis shows that actually:

$$n\lfloor \log(U/n) \rfloor + n + \lceil U/2^\ell \rceil \leq n(\log(U/n) + 2).$$

Interpretations of the high-bit encoding

- Each input number corresponds to a 1.
- Each "bucket" for the high parts corresponds to a 0.
- 0 separates the unary count in number of 1 for each high part.
- The number of 0 directly before a 1 is the jump the high part makes from the previous value.
- The total number of 0s before a 1 is the value of the high part.

Querying Elias-Fano

Definition 2.

To get x_i:

- Read the i'th group of ℓ low bits.

Querying Elias-Fano

Definition 2.

To get x_i:

- Read the i'th group of ℓ low bits.
- Get the high part of the i'th value:
  - Count zeros before the i'th 1 (counting from 0):

$$\mathsf{high}(i) = \mathsf{select}_1(i) - i$$

Variants

- As discussed: storing sorted lists of integers.

Variants

- As discussed: storing sorted lists of integers.
- For *strictly increasing* integers: replace x_i by $x_i - i$.

Variants

- As discussed: storing sorted lists of integers.
- For *strictly increasing* integers: replace x_i by $x_i - i$.
- To store a non-sorted list with small sum: store prefix sums.

Further reading / sources

- Giulio Pibiri's notes on Elias-Fano
- Modern paper reviving/applying it: (Vigna 2013)
- Original papers: (Elias 1974; Fano 1971)
- Dynamic version: (Pibiri and Venturini 2017)
- `sux::EliasFano` implementation

Bibliography

Elias, Peter. 1974. “Efficient Storage and Retrieval by Content and Address of Static Files.” *Journal of the Acm* 21 (2): 246–60. https://doi.org/10.1145/321812.321820.

Fano, R.M. 1971. *On the Number of Bits Required to Implement an Associative Memory*. Memorandum 61, Computation Structures Group. MIT Project MAC Computer Structures Group. https://books.google.ch/books?id=07DeGwAACAAJ.

Pibiri, Giulio Ermanno, and Rossano Venturini. 2017. “Dynamic Elias-Fano Representation.” In *Lipics, Volume 78, Cpm 2017*, 78:30:1–30:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. https://doi.org/10.4230/LIPICS.CPM.2017.30.

Vigna, Sebastiano. 2013. “Quasi-Succinct Indices.” In *Proceedings of the Sixth Acm International Conference on Web Search and Data Mining*, 83–92. Wsdm 2013. ACM. https://doi.org/10.1145/2433396.2433409.